

Sudoku Solver Notebook

Complete Cell-by-Cell Explanation

Computer Vision · KNN Digit Recognition · Backtracking Algorithm ·
SQLite

Pipeline Overview

Camera Image → Preprocess → Detect Board
→ Perspective Warp → KNN Digit Recognition
→ Backtracking Solver → Overlay Result → SQLite Storage

Cells 1 through 20 explained in full detail

Contents

1	Cell 1 — Install & Import Libraries	2
2	Cell 2 — SQLite Database Setup	3
3	Cell 3 — Database Helper Functions	4
4	Cell 4 — Image Preprocessing	4
5	Cell 5 — Grid Detection & Perspective Transform	6
6	Cell 6 — KNN Digit Classifier Training	7
7	Cell 7 — Digit Recognition from Cells	9
8	Cell 8 — Sudoku Solver (Optimised Backtracking)	10
9	Cell 9 — Overlay Solution on Original Image	11
10	Cell 10 — Pretty-Print Grid Helper	12
11	Cell 11 — Main Pipeline (<code>solve_from_image</code>)	13
12	Cell 12 — Generate Test Image	14
13	Cell 13 — Run the Full Pipeline	14
14	Cell 14 — Verify Digit Recognition Accuracy	15
15	Cell 15 — Database Statistics	15
16	Cell 16 — Random Puzzle Generator	16
17	Cell 17 — Query the Database	17
18	Cell 18 — Retrieve Stored Result Image	17
19	Cell 19 — Batch Processing	18
20	Cell 20 — Project Summary	18
	Appendix — Key OpenCV Functions Reference	19

1. Cell 1 — Install & Import Libraries

Cell 1

Purpose: Import all third-party and standard-library modules needed throughout the notebook.

```
1 import cv2
2 import numpy as np
3 import sqlite3
4 import os, time, json, base64
5 from copy import deepcopy
6 from datetime import datetime
7 import random
8 import matplotlib.pyplot as plt
9 import matplotlib.patches as patches
10 from sklearn.neighbors import KNeighborsClassifier
11 from IPython.display import display
```

Library-by-Library Explanation

- **cv2 (OpenCV)** — The core computer-vision library. Provides image I/O, colour-space conversion, contour detection, perspective transforms, morphological operations, and drawing functions. Every CV step in this notebook calls **cv2**.
- **numpy (np)** — N-dimensional array library. All images are stored as NumPy arrays (height × width × channels). Also used for fast set arithmetic when computing Sudoku candidates.
- **sqlite3** — Python's built-in SQLite driver. Used to persist puzzles, solutions, detection metadata, and result images without a separate database server.
- **os, time, json, base64** — Standard-library utilities: file existence checks, high-resolution timing (`time.time()`), JSON serialisation of grids, and Base64 encoding of binary PNG data for database storage.
- **deepcopy** — Used to clone the puzzle grid before solving, so the original (unsolved) grid is preserved for the overlay step.
- **random** — Supplies `random.shuffle()` for the randomised puzzle-generation algorithm.
- **matplotlib** — Renders intermediate visualisation steps and the final solved-puzzle image inside the Jupyter notebook.
- **KNeighborsClassifier** — Scikit-learn's K-Nearest Neighbours classifier. Trained on synthetic digit images to classify each Sudoku cell as 0–9.
- **display** — IPython helper that renders objects (images, HTML) inline in notebook output cells.

2. Cell 2 — SQLite Database Setup

Cell 2

Purpose: Create four SQLite tables that store every artefact produced by the pipeline, enabling full auditability and batch querying.

```

1 DB_PATH = 'sudoku_solver.db'
2
3 def get_connection():
4     conn = sqlite3.connect(DB_PATH)
5     conn.row_factory = sqlite3.Row
6     conn.execute('PRAGMA journal_mode=WAL')
7     conn.execute('PRAGMA foreign_keys=ON')
8     return conn
9
10 def init_database():
11     conn = get_connection()
12     c = conn.cursor()
13     c.execute('''CREATE TABLE IF NOT EXISTS puzzles (
14                 id INTEGER PRIMARY KEY AUTOINCREMENT,
15                 source_path TEXT, grid_json TEXT NOT NULL,
16                 empty_cells INTEGER NOT NULL, created_at TEXT NOT NULL)''')
17     # ... solutions, detection_logs, result_images tables ...
18     conn.commit(); conn.close()

```

Key Concepts

get_connection()

- `conn.row_factory = sqlite3.Row` — Makes each database row accessible by column name (`row['id']`) in addition to numeric index.
- `PRAGMA journal_mode=WAL` — Write-Ahead Logging: allows simultaneous reads and writes, improving performance when multiple pipeline runs overlap.
- `PRAGMA foreign_keys=ON` — Enforces referential integrity, e.g. a `solutions` row cannot reference a non-existent `puzzle_id`.

Database Schema

Table	Key Columns	What it Stores
puzzles	id, grid_json, empty_cells	Raw 9×9 grid from image or generated
solutions	puzzle_id, solve_time_ms, is_valid	Solved grid + timing
detection_logs	confidence, corners_json, processing_ms	CV pipeline metadata
result_images	solution_id, image_b64	Base64 PNG of annotated result

3. Cell 3 — Database Helper Functions

Cell 3

Purpose: Thin wrapper functions that handle all database INSERT and SELECT operations, keeping business logic separate from SQL.

```

1 def save_puzzle(grid, source_path=''):
2     empty = int(np.sum(np.array(grid) == 0))
3     conn = get_connection()
4     c.execute('INSERT INTO puzzles ... VALUES (?, ?, ?, ?)',
5               (source_path, json.dumps(grid), empty, datetime.now().isoformat()))
6     return c.lastrowid # returns the new puzzle ID
7
8 def save_solution(puzzle_id, solution, solve_ms, is_valid): ...
9 def save_detection_log(puzzle_id, ...): ...
10 def save_result_image(solution_id, img_array): ...
11 def fetch_history(limit=10): ...

```

Function Explanations

save_puzzle(grid, source_path) Counts empty cells with `np.sum(grid == 0)`, serialises the 9×9 list to JSON, and inserts a new row. Returns the auto-incremented id via `c.lastrowid` — this ID links all subsequent records together.

save_solution(puzzle_id, solution, solve_ms, is_valid) Stores the solved grid plus timing and a boolean validity flag (0 or 1, since SQLite has no boolean type).

save_detection_log(...) Records whether the board was detected, the confidence score, the 4 corner coordinates as JSON, image dimensions, and processing time. Invaluable for debugging failed detections.

save_result_image(solution_id, img_array) Encodes the NumPy image array as a PNG byte buffer (`cv2.imencode`), then Base64-encodes it so it can be stored as plain text in SQLite.

fetch_history(limit) Joins `puzzles` and `solutions` with a LEFT JOIN to retrieve the most recent *n* runs, including timing and validity.

4. Cell 4 — Image Preprocessing

Cell 4

Purpose: Convert a raw colour photograph into a clean binary image where grid lines appear as white pixels on a black background, ready for contour detection.

```

1 def preprocess_image(frame):
2     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
3     blurred = cv2.GaussianBlur(gray, (9, 9), 0)
4     binary = cv2.adaptiveThreshold(
5         blurred, 255,
6         cv2.ADAPTIVE_THRESH_GAUSSIAN_C,

```

```
7         cv2.THRESH_BINARY_INV,  
8         11, 2  
9     )  
10    kernel = np.ones((3, 3), np.uint8)  
11    binary = cv2.dilate(binary, kernel, iterations=1)  
12    return gray, blurred, binary
```

Computer Vision Deep Dive

Step 1 — Grayscale Conversion

$$\text{Gray}(x, y) = 0.114 \cdot B + 0.587 \cdot G + 0.299 \cdot R$$

OpenCV stores images in BGR order. Converting to grayscale reduces a 3-channel image to 1 channel, eliminating colour information that is irrelevant for a black-and-white puzzle.

Step 2 — Gaussian Blur

A 9×9 Gaussian kernel is convolved over the image:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

This smooths out paper grain, camera sensor noise, and ink texture. A larger kernel (9×9) produces stronger smoothing.

Step 3 — Adaptive Thresholding

Unlike a global threshold (one fixed value for the whole image), adaptive thresholding computes a separate threshold $T(x, y)$ for each pixel based on the local neighbourhood:

$$\text{Binary}(x, y) = \begin{cases} 255 & \text{if } \text{Gray}(x, y) < T(x, y) \\ 0 & \text{otherwise} \end{cases}$$

`ADAPTIVE_THRESH_GAUSSIAN_C` weights neighbouring pixels by a Gaussian window (block size = 11, constant subtracted = 2). `THRESH_BINARY_INV` inverts the result so dark grid lines become white. This handles uneven lighting across a real photograph robustly.

Step 4 — Dilation

A 3×3 structuring element is slid over the binary image. Any foreground pixel within 1 pixel of a white pixel is set to white, effectively *thickening* grid lines. This helps the contour finder connect slightly broken line segments.

5. Cell 5 — Grid Detection & Perspective Transform

Cell 5

Purpose: Locate the Sudoku board as the largest quadrilateral in the image, then “unfold” it into a flat 450×450 top-down view regardless of camera angle.

```

1 def find_largest_quadrangle(binary_image):
2     contours, _ = cv2.findContours(binary_image,
3                                     cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
4     best, max_area = None, 0
5     for cnt in contours:
6         area = cv2.contourArea(cnt)
7         if area < 5000: continue
8         peri = cv2.arcLength(cnt, True)
9         for eps in [0.02, 0.03, 0.04, 0.05, 0.06]:
10             approx = cv2.approxPolyDP(cnt, eps * peri, True)
11             if len(approx) == 4 and area > max_area:
12                 best, max_area = approx, area
13                 break
14     confidence = min(max_area / (w * h), 1.0) if best else 0.0
15     return best, confidence

```

Computer Vision Deep Dive

find_largest_quadrangle

Contour Detection. `cv2.findContours` traces the boundaries of all connected white regions in the binary image. `RETR_EXTERNAL` returns only outermost contours (ignores holes), and `CHAIN_APPROX_SIMPLE` compresses collinear points to save memory.

Douglas-Peucker Polygon Approximation. `cv2.approxPolyDP` simplifies a complex contour into a polygon with fewer vertices. The epsilon parameter ε controls how aggressively points are removed: points closer than $\varepsilon \times \text{perimeter}$ to the simplified line are eliminated. The code tries five values (2%–6% of perimeter) until it finds exactly 4 vertices — a quadrilateral.

Confidence Score. `confidence = board area / image area`. A score near 1.0 means the board fills most of the frame (ideal). Below ≈ 0.1 suggests a distant or misidentified board.

```

1 def reorder_corners(points):
2     pts = points.reshape(4, 2).astype(np.float32)
3     s = pts.sum(axis=1)
4     ordered[0] = pts[np.argmin(s)] # TL: smallest x+y
5     ordered[2] = pts[np.argmax(s)] # BR: largest x+y
6     diff = np.diff(pts, axis=1).flatten()
7     ordered[1] = pts[np.argmin(diff)] # TR: smallest x-y
8     ordered[3] = pts[np.argmax(diff)] # BL: largest x-y

```

reorder_corners

The four detected corner points can be in any order. This function sorts them into a canonical **TL**, **TR**, **BR**, **BL** order using the following identities:

$$\begin{aligned} \text{TL} &= \arg \min(x + y) & \text{BR} &= \arg \max(x + y) \\ \text{TR} &= \arg \min(x - y) & \text{BL} &= \arg \max(x - y) \end{aligned}$$

These hold for any convex quadrilateral in standard image coordinates.

```

1 def warp_board(frame, corners, target_size=450):
2     ordered = reorder_corners(corners)
3     dst = np.float32([[0,0],[450,0],[450,450],[0,450]])
4     M = cv2.getPerspectiveTransform(ordered, dst)
5     M_inv = cv2.getPerspectiveTransform(dst, ordered)
6     warped = cv2.warpPerspective(gray, M, (450, 450))
7     return warped, M, M_inv

```

warp_board — Perspective Transform

A perspective (homography) transform is a 3×3 matrix **M** that maps four source points to four destination points:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \mathbf{M} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (\text{homogeneous coordinates})$$

`cv2.getPerspectiveTransform` solves for **M** given the 4 detected corners (source) and the 4 corners of a 450×450 square (destination). `cv2.warpPerspective` applies **M** to every pixel, producing a flat top-down view. The inverse $\mathbf{M}^{-1}$ is kept for the overlay step.

6. Cell 6 — KNN Digit Classifier Training

Cell 6

Purpose: Build and train a K-Nearest Neighbours classifier on $\approx 3,900$ synthetically rendered digit images, producing a model that recognises handwritten-style digits 1–9 from Sudoku cells.

```

1 def build_knn_classifier():
2     fonts      = [cv2.FONT_HERSHEY_SIMPLEX, cv2.FONT_HERSHEY_DUPLEX]
3     scales     = [1.0, 1.1, 1.2, 1.3, 1.4, 1.5]
4     thicknesses = [1, 2, 3]
5     offsets    = [(0,0),(2,0),(-2,0),(0,2),(0,-2),(1,1),(-1,-1)]
6
7     for digit in range(1, 10):
8         for font in fonts:
9             for scale in scales:
10                for thick in thicknesses:
11                    for dx, dy in offsets:
12                        # render digit on 60x60 canvas
13                        # apply optional blur

```



```

14         # threshold -> find bounding box
15         # resize to 20x20 -> flatten -> append
16         clf = KNeighborsClassifier(n_neighbors=5, weights='distance')
17         clf.fit(X, y)
18         return clf, X, y

```

Computer Vision Deep Dive

Synthetic Data Generation

Rather than collecting real labeled images, every digit is *rendered* with `cv2.putText` across a parameter grid:

- **2 fonts** (SIMPLEX, DUPLEX) — different stroke styles
- **6 scales** (1.0–1.5) — simulates different zoom levels
- **3 thicknesses** (1–3) — simulates pen weight variation
- **7 position offsets** — simulates off-centre digits in cells
- **2 blur variants** (with/without Gaussian blur) — simulates focus

Total: $9 \times 2 \times 6 \times 3 \times 7 \times 2 = 4,536$ samples (minus any rejected by the area filter).

Feature Extraction Pipeline (per sample)

1. Threshold the 60×60 canvas with `THRESH_BINARY_INV` to get white digit on black background
2. Find contours; take the largest (the digit)
3. Crop to bounding box with `cv2.boundingRect`
4. Resize to 20×20 pixels
5. Flatten to a 400-element vector, normalise to $[0, 1]$

KNN Classifier

KNN classifies a new sample by finding its k nearest training samples (by Euclidean distance in the 400-dimensional feature space) and returning the majority class:

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_k(x)} \frac{\mathbf{1}[y_i = c]}{d(x, x_i)}$$

`weights='distance'` means closer neighbours contribute more to the vote. $k = 5$ balances bias and variance for this task.

7. Cell 7 — Digit Recognition from Cells

Cell 7

Purpose: Split the flat 450×450 board into 81 individual cell images and classify each one as either empty (0) or a digit (1–9).

```

1 def extract_digit_from_cell(cell_img, clf):
2     # 1. Crop inner region (remove grid-line border)
3     roi = cell_img[my:h-my, mx:w-mx]
4     # 2. Otsu threshold
5     _, thresh = cv2.threshold(roi, 0, 255,
6                               cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
7     # 3. Empty check: pixel density < 3%
8     if np.count_nonzero(thresh) / thresh.size < 0.03:
9         return 0
10    # 4. Find and filter contours
11    cnts = [c for c in cnts if cv2.contourArea(c) > 20]
12    # 5. Crop largest contour -> resize -> KNN predict
13    feat = cv2.resize(crop, (20, 20)).flatten() / 255.0
14    return int(clf.predict([feat])[0])
15
16 def read_grid_from_warped(warped_gray, clf):
17     cell_size = 450 // 9    # = 50 pixels
18     for r in range(9):
19         for c in range(9):
20             cell = warped_gray[r*50:(r+1)*50, c*50:(c+1)*50]
21             digit = extract_digit_from_cell(cell, clf)

```

Computer Vision Deep Dive

Border Cropping

Each 50×50 cell slice includes grid-line pixels at its edges. Cropping $\approx 1/6$ of the width and height on each side (approximately 8 pixels) removes these without cutting into the digit.

Otsu's Thresholding

Unlike the fixed threshold used during preprocessing, Otsu's method *automatically* finds the optimal global threshold for each individual cell by minimising intra-class variance:

$$\sigma_W^2(T) = \omega_0(T)\sigma_0^2(T) + \omega_1(T)\sigma_1^2(T)$$

where ω_0, ω_1 are the pixel-count fractions below/above T . This adapts well to cells that are brighter or darker than average.

Empty Cell Heuristic

If fewer than 3% of pixels in the thresholded ROI are white (foreground), the cell is declared empty. This threshold was tuned empirically to avoid false positives from cell-crossing grid lines.

read_grid_from_warped

A simple nested loop. Slicing is: `warped[r*50:(r+1)*50, c*50:(c+1)*50]`. Returns a 9×9 Python list where 0 = empty and 1–9 = recognised digit.

8. Cell 8 — Sudoku Solver (Optimised Backtracking)

Cell 8

Purpose: Solve the 9×9 puzzle using recursive backtracking accelerated by the **Most-Constrained-Variable (MCV)** heuristic.

```

1 def get_candidates(grid, row, col):
2     candidates = set(range(1, 10))
3     g = np.array(grid)
4     candidates -= set(g[row])           # remove row peers
5     candidates -= set(g[:, col])       # remove column peers
6     br, bc = (row//3)*3, (col//3)*3
7     candidates -= set(g[br:br+3, bc:bc+3].flatten()) # box peers
8     return candidates
9
10 def solve_sudoku(grid):
11     min_cell, min_count = None, 10
12     for i in range(9):
13         for j in range(9):
14             if grid[i][j] == 0:
15                 cands = get_candidates(grid, i, j)
16                 if len(cands) == 0: return False # dead end
17                 if len(cands) < min_count:
18                     min_count = len(cands)
19                     min_cell = (i, j, cands)
20     if min_cell is None: return True # all filled
21     row, col, cands = min_cell
22     for num in cands:
23         grid[row][col] = num
24         if solve_sudoku(grid): return True
25         grid[row][col] = 0 # backtrack
26     return False

```

Algorithm Deep Dive: Backtracking with MCV Heuristic

get_candidates

Uses NumPy set arithmetic to compute the legal values for cell (r, c) :

1. Start with $\{1, 2, \dots, 9\}$
2. Remove every digit already in row r : `g[row]`
3. Remove every digit already in column c : `g[:, col]`
4. Remove every digit in the enclosing 3×3 box: `g[br:br+3, bc:bc+3].flatten()`

The remaining set contains exactly the values that could legally occupy (r, c) given the current partial assignment.

solve_sudoku — Algorithm

1. **Scan** all empty cells and compute their candidate sets.
2. **Dead-end check**: if any cell has 0 candidates, the current assignment is unsolvable — return **False** immediately to trigger backtracking in the parent call.
3. **Termination**: if no empty cells remain, the puzzle is solved — return **True**.
4. **MCV selection**: choose the empty cell with the *fewest* candidates. By branching on the most constrained variable first, the algorithm detects contradictions earlier and prunes more branches.
5. **Try each candidate**: place a number, recurse.
 - If the recursive call succeeds, propagate **True** upward.
 - If it fails, **backtrack**: reset `grid[row][col] = 0` and try the next candidate.
6. If all candidates fail, return **False** to backtrack further.

Performance Impact

Strategy	Typical Solve Time	Speedup
Naive (fixed order)	600–2300 ms	1×
MCV heuristic	≈13 ms	~100×

validate_solution

After solving, every row, column, and 3×3 box is checked to contain exactly the set $\{1, \dots, 9\}$ using **set** equality:

```

1 def validate_solution(solution):
2     g = np.array(solution)
3     expected = set(range(1, 10))
4     for i in range(9):
5         if set(g[i]) != expected or set(g[:, i]) != expected:
6             return False
7     for br in range(0, 9, 3):
8         for bc in range(0, 9, 3):
9             if set(g[br:br+3, bc:bc+3].flatten()) != expected:
10                 return False
11     return True

```

9. Cell 9 — Overlay Solution on Original Image**Cell 9**

Purpose: Draw the newly solved digits (green) back onto the original camera-perspective photograph using inverse perspective warping.

```

1 def overlay_solution(frame, puzzle, solution, corners):
2     canvas = np.zeros((450, 450, 3), dtype=np.uint8)
3     for r in range(9):

```

```

4     for c in range(9):
5         if puzzle[r][c] == 0:      # only cells that were empty
6             cv2.putText(canvas, str(solution[r][c]),
7                           (x, y), font, font_scale, (0,210,60), thickness)
8     # Inverse perspective warp: flat canvas -> original perspective
9     M_back = cv2.getPerspectiveTransform(src_pts, dst_pts)
10    warped_canvas = cv2.warpPerspective(canvas, M_back,
11                                       (frame.shape[1], frame.shape[0]))
12    mask = warped_canvas.sum(axis=2) > 0
13    result[mask] = warped_canvas[mask]
14    return result

```

How the Overlay Works

1. A blank 450×450 black canvas is created in the “flat” coordinate space.
2. Only cells that were *originally empty* (`puzzle[r][c] == 0`) receive a drawn digit. Pre-filled clues are left untouched.
3. A digit is centred in each cell using `cv2.getTextSize` to measure the text bounding box.
4. The canvas is inverse-perspective-warped using `cv2.getPerspectiveTransform` with source = 450×450 square, destination = the original 4 detected corners. This maps the flat digits back into the tilted, foreshortened space of the original photo.
5. A pixel mask (`sum(axis=2) > 0`) selects only non-black pixels in the warped canvas and composites them onto the original frame.

10. Cell 10 — Pretty-Print Grid Helper

Cell 10

Purpose: Print a 9×9 grid to the terminal with Unicode box-drawing characters, showing 3×3 box boundaries.

```

1 def print_grid(grid, title='Grid'):
2     print(' +-----+-----+-----+')
3     for r, row in enumerate(grid):
4         if r in (3, 6): print(' +-----+-----+-----+')
5         line = ' |'
6         for c, val in enumerate(row):
7             line += f" {'.' if val == 0 else val}"
8             if c in (2, 5): line += ' |'
9         print(line + ' |')
10    print(' +-----+-----+-----+')

```

The function uses row dividers at indices 3 and 6, and column dividers at indices 2 and 5, to visually separate the nine 3×3 boxes. Empty cells display as dots.

11. Cell 11 — Main Pipeline (solve_from_image)

Cell 11

Purpose: Orchestrate all pipeline steps in sequence, log every artefact to SQLite, and optionally display intermediate visualisations.

```

1 def solve_from_image(image_path, manual_puzzle=None,
2                       save_to_db=True, show_steps=True):
3     # [1/8] Load image
4     frame = cv2.imread(image_path)
5     # [2/8] Preprocess
6     gray, blurred, binary = preprocess_image(frame)
7     # [3/8] Detect board
8     corners, confidence = find_largest_quadrangle(binary)
9     # [4/8] Recognise digits (or use manual_puzzle)
10    warped, _, _ = warp_board(frame, corners)
11    puzzle = read_grid_from_warped(warped, KNN_CLF)
12    # [5/8] Save puzzle
13    puzzle_id = save_puzzle(puzzle, source_path=image_path)
14    # [6/8] Solve
15    solution = deepcopy(puzzle)
16    success = solve_sudoku(solution)
17    # [7/8] Save solution
18    solution_id = save_solution(puzzle_id, solution, solve_ms, is_valid)
19    # [8/8] Overlay & display
20    result_img = overlay_solution(frame, puzzle, solution, corners)
21    save_result_image(solution_id, result_img)

```

Pipeline Steps

Step	Function Called	What Happens
1/8	<code>cv2.imread</code>	Load JPEG/PNG from disk into BGR array
2/8	<code>preprocess_image</code>	Gray → blur → threshold → dilate
3/8	<code>find_largest_quadrangle</code>	Detect 4 corners of the board
4/8	<code>warp_board</code> + <code>read_grid_from_warped</code>	Flatten + recognise 81 digits
5/8	<code>save_puzzle</code>	Persist raw grid to SQLite
6/8	<code>solve_sudoku</code>	Backtracking solver
7/8	<code>save_solution</code>	Persist solved grid + timing
8/8	<code>overlay_solution</code> + <code>save_result_image</code>	Composite and store result

The `manual_puzzle` parameter lets users bypass all CV steps and provide a hand-typed grid, useful for testing the solver in isolation.

12. Cell 12 — Generate Test Image

Cell 12

Purpose: Render a synthetic Sudoku image with optional perspective distortion, enabling a full end-to-end demo without a camera or real photograph.

```

1 def generate_test_image(puzzle, output_path='test_sudoku.jpg', tilt=True):
2     cell, pad = 60, 20
3     img = np.ones((9*60+40, 9*60+40, 3), np.uint8) * 255
4     # Draw outer border & grid lines (thick at multiples of 3)
5     for i in range(10):
6         thick = 3 if i % 3 == 0 else 1
7         cv2.line(img, ...)
8     # Draw digits
9     for r in range(9):
10        for c in range(9):
11            if puzzle[r][c] != 0:
12                cv2.putText(img, str(puzzle[r][c]), ...)
13    if tilt:
14        src = np.float32([[0,0],[w,0],[w,h],[0,h]])
15        dst = np.float32([[10,5],[w-5,0],[w-8,h-3],[5,h]])
16        M = cv2.getPerspectiveTransform(src, dst)
17        img = cv2.warpPerspective(img, M, (w, h))
18    cv2.imwrite(output_path, img)

```

Key Details

- Grid lines at multiples of 3 use thickness 3; inner lines use thickness 1, replicating the visual appearance of a printed Sudoku grid.
- The `tilt` option applies a mild perspective transform (shifting corners by 5–10 pixels) to simulate the slight angle of a hand-held photo. This exercises the full CV detection pipeline rather than processing a perfectly aligned image.

13. Cell 13 — Run the Full Pipeline

Cell 13

Purpose: Entry point — configure the image path and call `solve_from_image` to execute the complete pipeline.

```

1 IMAGE_PATH    = TEST_IMAGE_PATH    # replace with your own photo path
2 MANUAL_PUZZLE = None                # or provide a 9x9 list
3
4 result = solve_from_image(
5     image_path    = IMAGE_PATH,
6     manual_puzzle = MANUAL_PUZZLE,
7     save_to_db    = True,
8     show_steps    = True,
9 )

```

This is the cell a user re-runs with their own Sudoku photograph. Set `MANUAL_PUZZLE` to a 9×9 nested list to skip CV recognition (useful when the image quality is too poor for reliable detection).

14. Cell 14 — Verify Digit Recognition Accuracy

Cell 14

Purpose: Compare the KNN-recognised puzzle against a known ground truth and compute per-cell accuracy, highlighting any misclassified cells.

```

1 ground_truth = DEMO_PUZZLE
2 correct, total = 0, 0
3 errors = []
4 for r in range(9):
5     for c in range(9):
6         if ground_truth[r][c] != 0:    # only check pre-filled cells
7             total += 1
8             if result['puzzle'][r][c] == ground_truth[r][c]:
9                 correct += 1
10            else:
11                errors.append((r, c, ground_truth[r][c], result['puzzle'][r][c]))
12 acc = correct / total * 100
13 print(f'Digit Recognition Accuracy: {correct}/{total} = {acc:.1f}%')
```

Only pre-filled (non-zero) cells are compared, since empty cells are by definition correct (returned as 0 by `extract_digit_from_cell`). The cell also prints `solve_ms` and `is_valid` from the result dict.

15. Cell 15 — Database Statistics

Cell 15

Purpose: Query aggregate statistics across all stored runs, showing solve-time distribution and success rate.

```

1 def show_stats():
2     stats = conn.execute('''
3         SELECT
4             COUNT(*)                AS total_puzzles,
5             SUM(s.is_valid)          AS solved_ok,
6             ROUND(AVG(s.solve_time_ms),2) AS avg_ms,
7             ROUND(MIN(s.solve_time_ms),2) AS min_ms,
8             ROUND(MAX(s.solve_time_ms),2) AS max_ms,
9             ROUND(AVG(p.empty_cells),1) AS avg_empty
10            FROM puzzles p LEFT JOIN solutions s ON s.puzzle_id = p.id
11    ''').fetchone()
```

Uses a `LEFT JOIN` so puzzles with no associated solution (e.g. a detection failure) still appear in the count. SQL aggregates (`AVG`, `MIN`, `MAX`) compute timing statistics directly in the database without loading data into Python.

16. Cell 16 — Random Puzzle Generator

Cell 16

Purpose: Generate brand-new random Sudoku puzzles at configurable difficulty, solve them, visualise both states, and persist everything to SQLite.

```

1 CLUE_COUNT = {"easy": 36, "medium": 30, "hard": 24}
2
3 def _fill_grid(grid):
4     """Recursively fill a valid Sudoku solution (randomised)."""
5     for row in range(9):
6         for col in range(9):
7             if grid[row][col] == 0:
8                 nums = list(range(1, 10))
9                 random.shuffle(nums)          # randomise candidate order
10                for num in nums:
11                    if valid_placement:
12                        grid[row][col] = num
13                        if _fill_grid(grid): return True
14                        grid[row][col] = 0
15                return False
16        return True
17
18 def generate_puzzle(difficulty="hard"):
19     solution = [[0]*9 for _ in range(9)]
20     _fill_grid(solution)
21     cells_to_remove = 81 - CLUE_COUNT[difficulty]
22     puzzle = deepcopy(solution)
23     for pos in random.sample(range(81), cells_to_remove):
24         r, c = divmod(pos, 9)
25         puzzle[r][c] = 0
26     return puzzle, solution

```

`_fill_grid` — Randomised Backtracking

Uses the same backtracking algorithm as `solve_sudoku`, but shuffles the candidate list with `random.shuffle` before iterating. This produces a different valid complete grid on each call, forming the basis for puzzle generation.

`generate_puzzle` — Difficulty

Difficulty	Clues Kept	Cells Removed
Easy	36	45
Medium	30	51
Hard	24	57

Cells are removed at random without uniqueness verification (a production solver would ensure only one solution exists).

render_sudoku_image

Draws the Sudoku grid using Matplotlib's `patches.Rectangle` and `ax.text`, colouring given clues dark and solved digits green. Converts the figure to a NumPy BGR array via the canvas buffer for storage.

17. Cell 17 — Query the Database

Cell 17

Purpose: Display a formatted table of the 10 most recent pipeline runs directly from SQLite.

```

1 history = fetch_history(limit=10)
2 print(f"{'ID':<5} {'Source':<30} {'Empty':<7} {'SolveMs':<10} {'Valid'}")
3 for row in history:
4     ms = f"{row['solve_time_ms']:.1f}" if row['solve_time_ms'] else 'N/A'
5     valid = 'v' if row['is_valid'] else 'x'
6     print(f"{'row[id]':<5} {src:<30} {row['empty_cells']:<7} {ms:<10} {valid}")

```

Uses Python f-string alignment specifiers (<5, <30) for column-aligned terminal output. The source path is truncated to 29 characters to fit the table.

18. Cell 18 — Retrieve Stored Result Image

Cell 18

Purpose: Decode a Base64-encoded PNG from the database and display the annotated result image inside the notebook.

```

1 def show_stored_result(solution_id):
2     row = conn.execute(
3         'SELECT image_b64 FROM result_images WHERE solution_id=?',
4         (solution_id,)).fetchone()
5     img_bytes = base64.b64decode(row['image_b64'])
6     buf = np.frombuffer(img_bytes, np.uint8)
7     img = cv2.imdecode(buf, cv2.IMREAD_COLOR)
8     plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
9     plt.show()

```

The round-trip is:

NumPy array $\xrightarrow{\text{cv2.imencode}}$ PNG bytes $\xrightarrow{\text{base64.b64encode}}$ ASCII string (DB) $\xrightarrow{\text{base64.b64decode}}$ PNG bytes $\xrightarrow{\text{cv2}}$

19. Cell 19 — Batch Processing

Cell 19

Purpose: Process multiple Sudoku images in a loop, storing all results and printing a summary.

```

1 def batch_solve(image_paths, manual_puzzles=None):
2     results = []
3     for i, path in enumerate(image_paths):
4         puzzle = manual_puzzles[i] if manual_puzzles else None
5         try:
6             r = solve_from_image(path, manual_puzzle=puzzle,
7                                   save_to_db=True, show_steps=False)
8             results.append(r)
9         except Exception as e:
10            results.append(None)
11    ok = sum(1 for r in results if r and r['is_valid'])
12    print(f'Batch done: {ok}/{len(image_paths)} solved')
13    return results

```

Sets `show_steps=False` to suppress per-image matplotlib output in batch mode. Uses a try/except per image so a single failure does not abort the entire batch. The final summary calls `show_stats()` to print aggregate DB metrics.

20. Cell 20 — Project Summary

Cell 20

Purpose: A reference diagram and schema summary for the entire project.

```

Camera Image
|
v
Preprocess  ->  grayscale -> GaussianBlur -> adaptiveThreshold -> dilate
|
v
Board Detection  ->  findContours -> approxPolyDP -> largest quad
|
v
Perspective Transform  ->  reorder corners -> warpPerspective -> 450x450
|
v
Digit Recognition  ->  split 81 cells -> Otsu -> bounding box
|                      -> resize 20x20 -> KNN (k=5, distance-weighted)
v
Backtracking Solver  ->  MCV heuristic -> validate
|
+--> SQLite DB -> puzzles | solutions | detection_logs | result_images
|
v
Overlay  ->  draw on canvas -> inverse warpPerspective -> composite

```

This cell serves as documentation only — it contains no executable logic, but provides a complete mental map of how all previous cells interconnect.

Appendix — Key OpenCV Functions Reference

Function	Description
<code>cv2.cvtColor</code>	Convert between colour spaces (BGR↔GRAY, BGR↔RGB)
<code>cv2.GaussianBlur</code>	2D Gaussian convolution for noise reduction
<code>cv2.adaptiveThreshold</code>	Local-neighbourhood binarisation
<code>cv2.dilate</code>	Morphological dilation — expands foreground regions
<code>cv2.findContours</code>	Trace boundaries of connected foreground regions
<code>cv2.contourArea</code>	Compute the area of a contour polygon
<code>cv2.arcLength</code>	Compute the perimeter of a contour
<code>cv2.approxPolyDP</code>	Douglas-Peucker polygon simplification
<code>cv2.getPerspectiveTransform</code>	Compute 3×3 homography from 4 point pairs
<code>cv2.warpPerspective</code>	Apply perspective transform to an image
<code>cv2.threshold (Otsu)</code>	Automatically find optimal global threshold
<code>cv2.boundingRect</code>	Axis-aligned bounding box of a contour
<code>cv2.resize</code>	Rescale image to target dimensions
<code>cv2.putText</code>	Render text string onto an image
<code>cv2.imencode</code>	Encode NumPy array as PNG/JPEG byte buffer
<code>cv2.imdecode</code>	Decode byte buffer back to NumPy array